

Text-Conditioned Image Generation via *Flow Matching*

Model Architecture and Results on Colored MNIST

Technical Report

2026

Abstract

This report presents a text-to-image generative model trained end-to-end: a cross-attention conditioned *U-Net* guided by a Transformer text encoder, optimized with the **flow matching** objective (a simple and efficient alternative to denoising diffusion probabilistic models). The model is trained on a synthetic image-captioning task built from colored, rotated digits. We present the mathematical formulation of flow matching, the architecture of the network, and the empirical results obtained from a full training run (loss curves, denoising trajectories, generated samples).

Contents

1	Introduction	1
2	Model Architecture	2
2.1	Text Encoder	2
2.2	Time Embedding	3
2.3	Conditioned U-Net	3
3	Flow Matching Formulation	4
3.1	General Principle	4
3.2	Interpolation Paths	4
3.3	Training Objective	5
3.4	Sampling (Inference)	5
4	Results	6
4.1	Loss Convergence	6
4.2	Sampling Trajectory (Denoising)	6
4.3	Generated Samples	7
4.4	Qualitative Analysis	7
5	Conclusion	8

1 Introduction

The goal of this work is to study, at a small and tractable scale, the core mechanisms behind modern text-to-image diffusion/flow models (Stable Diffusion, Imagen, etc.): a text encoder that turns a natural-language caption into a conditioning signal, and an image generator that turns that conditioning signal, together with a continuous notion of time, into a plausible image.

The task chosen for experimentation is generating colored, rotated handwritten digits from a free-form textual description of their color, digit identity, and rotation angle, for example a caption such as

"a red 7 rotated 90 degrees".

Three components are involved:

1. a text encoder that maps a caption to a sequence of contextual embeddings (Section 2);
2. an image generator conditioned on that text context via cross-attention (Section 2);
3. a *flow matching* training objective, a simple and effective alternative to classical DDPM training (Section 3).

2 Model Architecture

The model combines two sub-networks: a **text encoder** (Transformer) producing a sequential context, and a conditioned **U-Net** that predicts, at every timestep t , the flow matching velocity field (Section 3).

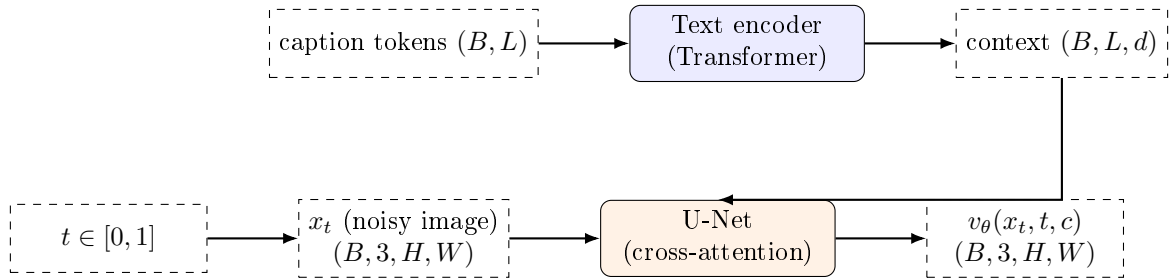


Figure 1: Model overview: the text encoder produces a sequential context $c = (B, L, d)$ consumed via cross-attention inside the U-Net, which predicts the velocity field $v_\theta(x_t, t, c)$ from the noisy image x_t and the timestep t .

2.1 Text Encoder

The caption is first mapped to a fixed-length sequence of token indices from a small closed vocabulary (describing color, digit, and rotation angle), and padded to a maximum sequence length. This token sequence is then encoded into a dense contextual representation by:

1. an embedding layer mapping each token index to a vector in $\mathbb{R}^d$;
2. a learned positional encoding, added elementwise;
3. a stack of standard Transformer encoder layers (self-attention + feed-forward, GELU activation, multi-head attention), which ignore padding positions via an attention mask;
4. a final layer normalization.

The output is a context $c \in \mathbb{R}^{B \times L \times d}$ — a *per-token* representation rather than a single pooled vector — which allows the image generator to perform token-level cross-attention, similarly to how Stable Diffusion conditions on CLIP text embeddings.

2.2 Time Embedding

The continuous time variable $t \in [0, 1]$ is embedded with a sinusoidal embedding (as in Transformers / DDPM):

$$e(t) = [\sin(tf_1), \dots, \sin(tf_{d/2}), \cos(tf_1), \dots, \cos(tf_{d/2})], \quad f_k = \exp\left(-k \cdot \frac{\ln(10000)}{d/2 - 1}\right),$$

then refined by a small multilayer perceptron (linear $\rightarrow$ SiLU $\rightarrow$ linear, with an expansion factor). The resulting embedding t_{emb} is injected into every residual block of the U-Net, giving the network access to the current position along the flow.

2.3 Conditioned U-Net

The image generator follows a classical encoder–bottleneck–decoder U-Net layout with skip connections, operating over multiple spatial resolutions:

- **Residual block:** group normalization $\rightarrow$ SiLU $\rightarrow 3 \times 3$ convolution, injection of the projected time embedding t_{emb} between the two convolutions, then group normalization $\rightarrow$ SiLU $\rightarrow 3 \times 3$ convolution again, plus a residual connection (with a 1×1 convolution when the number of channels changes).
- **Downsampling path:** residual blocks followed by stride-2 convolutions that halve the spatial resolution at each stage while doubling the channel count.
- **Bottleneck:** residual block $\rightarrow$ **cross-attention** $\rightarrow$ residual block, at the lowest spatial resolution, where the quadratic cost of attention (in the number of spatial positions) remains small.
- **Upsampling path:** transposed convolutions to double the resolution, concatenation with the skip connection from the symmetric downsampling stage, then a residual block to fuse the two.
- **Output head:** group normalization $\rightarrow$ SiLU $\rightarrow 3 \times 3$ convolution back to the image’s channel count (predicting the velocity field, with the same shape as the input image).

Cross-attention block. This block conditions the spatial feature map on the text context: the feature map $h \in \mathbb{R}^{B \times C \times H \times W}$ is flattened into a sequence of pseudo-tokens (B, HW, C) , which queries the text context $c \in \mathbb{R}^{B \times L \times d}$ (used as keys and values) via multi-head attention — the same mechanism used in Stable Diffusion to condition a U-Net on CLIP text embeddings. The attention output is added back (residual connection) to the feature map before the block’s second convolution.

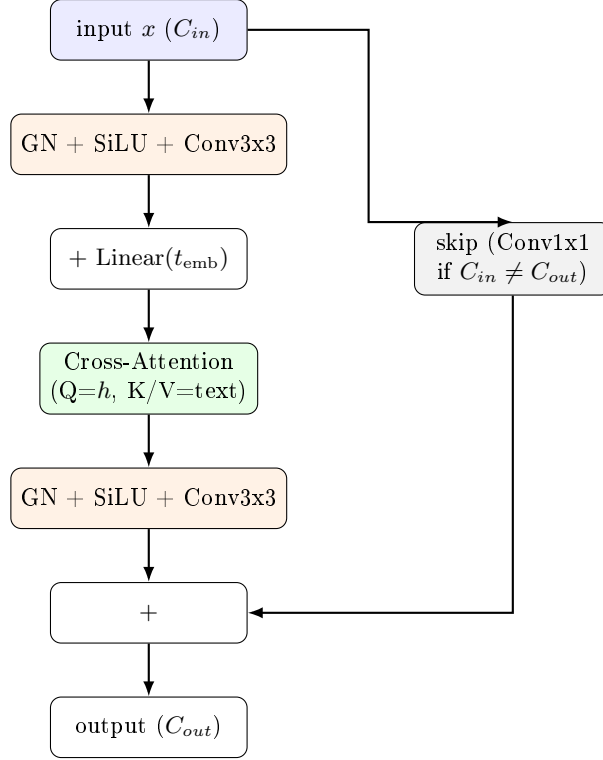


Figure 2: Detail of a cross-attention residual block: the time embedding is injected between the two convolutions (as in a plain residual block), and cross-attention on the text context is applied before the second convolution.

The resulting network is deliberately compact ($\approx 0.5\text{M}$ parameters) compared to production-scale diffusion U-Nets (often $> 100\text{M}$ parameters), trading generation fidelity for fast training on consumer-grade hardware while retaining the essential mechanisms: time conditioning, text cross-attention, and multi-scale skip connections.

3 Flow Matching Formulation

3.1 General Principle

Flow matching [1] learns a velocity field $v_\theta(x_t, t)$ that transports, along continuous time $t \in [0, 1]$, a noise sample $x_0 \sim \mathcal{N}(0, I)$ towards a data sample $x_1 \sim p_{\text{data}}$, following an ordinary differential equation (ODE):

$$\frac{dx_t}{dt} = v_\theta(x_t, t), \quad x_0 \sim \mathcal{N}(0, I) \rightarrow x_1 \approx \text{image}.$$

Unlike DDPMs (a stochastic process requiring hundreds of denoising steps), flow matching directly learns the velocity field of a probability path chosen *a priori* between noise and data, which makes training particularly simple: for each training example, a time t and a noise sample x_0 are drawn, the interpolated point x_t and its target velocity are computed in closed form, and the network is trained to regress that target velocity.

3.2 Interpolation Paths

Two interpolation paths between x_0 (noise) and x_1 (real data) are considered:

Linear (rectified flow) Linear interpolation:

$$x_t = (1 - t)x_0 + tx_1, \quad v^*(x_t, t) = \frac{dx_t}{dt} = x_1 - x_0,$$

the target velocity is **constant in t** for a given pair (x_0, x_1) : this is the *rectified flow* path [2], the simplest possible choice (a straight line in image space).

Cosine Trigonometric interpolation:

$$x_t = \cos\left(\frac{\pi t}{2}\right)x_0 + \sin\left(\frac{\pi t}{2}\right)x_1, \quad v^*(x_t, t) = \frac{\pi}{2} \left(\cos\left(\frac{\pi t}{2}\right)x_1 - \sin\left(\frac{\pi t}{2}\right)x_0 \right),$$

a path along an arc, whose target velocity depends explicitly on t (accelerating/decelerating near the endpoints).

3.3 Training Objective

At each training step, given a batch of real images x_1 (normalized to $[-1, 1]$) and their tokenized captions c :

1. sample noise $x_0 \sim \mathcal{N}(0, I)$ with the same shape as x_1 ;
2. sample a time $t \sim \mathcal{U}(0, 1)$, independently per example;
3. compute x_t and the target velocity v^* along the chosen path;
4. compute the predicted velocity $v_\theta(x_t, t, c)$ with the network;
5. minimize the mean squared error:

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1), x_0 \sim \mathcal{N}(0,I), (x_1, c) \sim p_{\text{data}}} \left[\|v_\theta(x_t, t, c) - v^*(x_t, t)\|_2^2 \right].$$

This is a plain MSE regression, with no additional weighting term or KL divergence — the main source of simplicity of flow matching compared to the ELBO-based training of DDPMs.

3.4 Sampling (Inference)

Image generation consists of numerically integrating the learned ODE, starting from pure noise, using the explicit Euler method with a constant step size $\Delta t = 1/N$ over N steps:

$$\begin{aligned} x^{(0)} &\sim \mathcal{N}(0, I) \\ x^{(i+1)} &= x^{(i)} + \Delta t \cdot v_\theta(x^{(i)}, i\Delta t, c), \quad i = 0, \dots, N-1 \\ \text{final image} &= \text{clip}(x^{(N)}, -1, 1) \text{ rescaled to } [0, 1]. \end{aligned}$$

Algorithm — Euler Integration Sampling

Input: model v_θ , caption context c , number of steps N

1. $x \leftarrow$ Gaussian noise $\sim \mathcal{N}(0, I)$, image-shaped
 2. $\Delta t \leftarrow 1/N$
 3. **for** $i = 0$ **to** $N - 1$ **do**
 - (a) $t \leftarrow i \cdot \Delta t$
 - (b) $v \leftarrow v_\theta(x, t, c)$ // one network forward pass
 - (c) $x \leftarrow x + v \cdot \Delta t$ // Euler step
 4. **return** $\text{clip}(x, -1, 1)$ rescaled to $[0, 1]$
-

Figure 3: Euler integration sampling procedure.

The partial image can be recovered after every intermediate step, which is useful to visualize the denoising trajectory (Figure 5) or to drive an interactive generation interface.

4 Results

This section reports results from a complete training run, and samples generated from the final trained model.

4.1 Loss Convergence

Figure 4 shows the flow matching MSE loss over the course of training. The raw per-batch loss (light blue) is overlaid with the per-epoch average loss (red).

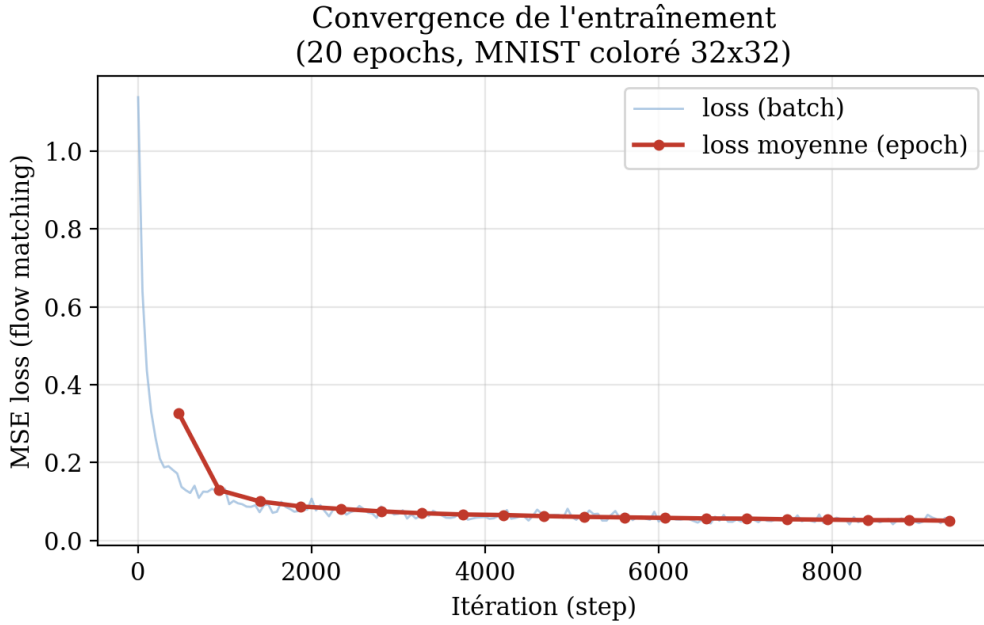


Figure 4: Flow matching loss (MSE) over the full training run. The loss drops sharply within the first epoch ($1.14 \rightarrow 0.33$) and then converges monotonically and smoothly down to ≈ 0.051 .

Table 1: Average loss per epoch (representative checkpoints).

Epoch	1	2	4	8	12	16	20
Loss	0.3273	0.1295	0.0877	0.0657	0.0583	0.0537	0.0510

This is the typical behavior of flow matching training: a very fast decrease over the first few hundred iterations (the model first learns the global statistics of the signal — background, dominant hue — the "easy" part of the target), followed by a slower refinement phase where the model sharpens the digit's shape and orientation. The curve shows no sign of instability (no divergence, no oscillation), which validates the choice of the linear interpolation path and learning rate for this model size.

4.2 Sampling Trajectory (Denoising)

Figure 5 illustrates the ODE integration process described in Section 3: starting from pure Gaussian noise, the trained model progressively transforms x_t into a coherent image, for the

caption "a red 7 rotated 90 degrees".

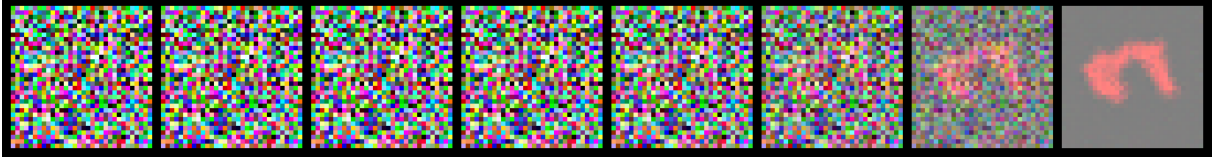


Figure 5: Denoising trajectory for the caption "a red 7 rotated 90 degrees", 50-step Euler integration. From left to right: initial noise ($t = 0$), intermediate steps ($t \approx 0.02$ to $t \approx 0.70$), and the final image ($t = 1$). Shape and color progressively emerge as $t \rightarrow 1$.

4.3 Generated Samples

Figure 6 presents a grid of 12 images generated by the final trained model, for 12 different combinations of color, digit, and rotation angle, using 50 integration steps.

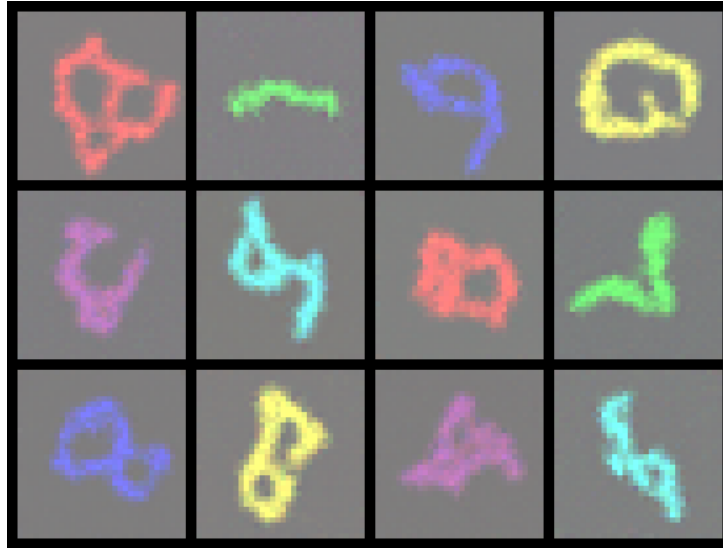


Figure 6: Samples generated by the trained model, for varied (color, digit, angle) captions. The model correctly learns the **requested color** and an overall **silhouette** evocative of a rotated digit, though fine stroke detail remains limited — consistent with the very small size of the network ($\approx 0.5M$ parameters) and the limited training duration.

4.4 Qualitative Analysis

- **Color:** consistently respected — the easiest signal to learn (it maps almost bijectively to a single color token in the caption), consistent with the rapid loss decrease within the first epoch.
- **Digit shape / identity:** partially learned — samples show loops and strokes consistent with MNIST digits, without clearly reproducing all ten digit classes; this is expected for such a small model trained for a limited number of steps.
- **Rotation:** generated silhouettes show varied orientations consistent with the angle conditioning, although fine-grained control of the exact angle is not visually verifiable at this resolution.
- **Sampling stability:** no ODE divergence was observed over the 50 integration steps; the trajectory stays well-behaved and the output remains within $[-1, 1]$ after clipping, prior to the final affine rescaling to $[0, 1]$.

These results are consistent with expectations for a small-scale model: the joint distribution over color, shape, and angle starts to be captured after a modest training budget on a sub-million-parameter network, which validates the flow-matching plus cross-attention pipeline end to end.

5 Conclusion

This report presented a complete text-conditioned image generation model, trained with the flow matching objective on a colored, rotated digit generation task. The architecture — a Transformer text encoder combined with a cross-attention U-Net — faithfully follows the design principles of production text-to-image diffusion models, at a scale suited to local experimentation. Training converges stably (loss $1.14 \rightarrow 0.051$) and produces samples that correctly respect the requested color while sketching the digit’s shape and orientation, validating the full pipeline from text conditioning to iterative ODE-based sampling.

References

- [1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le, *Flow Matching for Generative Modeling*, ICLR 2023.
- [2] X. Liu, C. Gong, Q. Liu, *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*, ICLR 2023.
- [3] J. Ho, A. Jain, P. Abbeel, *Denoising Diffusion Probabilistic Models*, NeurIPS 2020.
- [4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, *High-Resolution Image Synthesis with Latent Diffusion Models*, CVPR 2022.
- [5] A. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.